

Model Context Protocol Integration with File System

WPI Capstone Experience Proposal

Hengyu Liu, Joshua Joubert, and Maurisa Dacosta

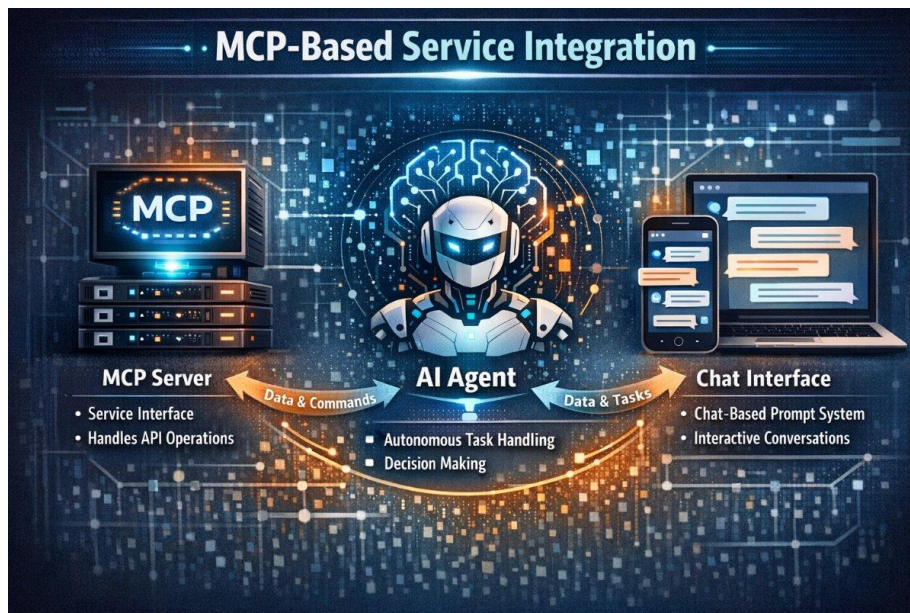
Revision Date: 02/11/2026

CS 587 MASTER OF SCIENCE CYBER SECURITY
CS 588 MASTER OF COMPUTER SCIENCE

Project Introduction

The goal of this project is to create a system that allows an AI agent to safely interact with a local file system using the Model Context Protocol (MCP). The system will consist of an MCP server that provides controlled access to file system operations, an AI agent that can perform file related tasks, and a chat-based interface that allows users to interact with the agent using natural language.

Many current AI tools require users to manually locate and upload files before the system can be helpful. This project aims to reduce that extra work by allowing an AI agent to serve as a glue between the LLM and the users' local drive, such as searching for files, uploading file to the model, and saving the model generated files to the local drive. All file operations will be restricted to a specific directory to prevent unauthorized access. The result will be a working prototype that demonstrates how MCP can be used to connect AI agents to real system resources in a safe and structured way.



Need / Significance

Many common tasks require working with files stored on a computer. Most AI tools can only use files that users manually upload, which makes these tasks slower and more frustrating, especially for students and professionals who work with documents and forms. A common example is filling out a

form using information already saved in local files. Users often must search for files, open them, and copy information by hand. An AI agent could help by finding the files and using the information automatically, but giving an AI full access to the file system is not safe. Another example to demonstrate the convenience of the system can provide is the bridge between the real-time terminal outputs and the AI model's interpretation, which makes the debugging process more streamline rather than constantly exchange their perspective outputs.

This project uses the Model Context Protocol to give an AI agent limited access to the file system. The idea is that the agent can only work inside a specific folder, and any file changes require user approval. This allows the system to work with existing file systems without replacing them. Overall, this project shows a simple and safe way for AI to help with file-based tasks using natural language. The work is meaningful because it shows how AI can be useful in everyday workflows while keeping users in control.

Project Objectives

The objective of this project is to build a working system that allows an AI agent to safely interact with a local file system using the Model Context Protocol.

Our goals are to:

- Create an MCP server that provides controlled access to file system operations such as searching, reading, and writing files.
- Use AI agent to help perform file related tasks through MCP tools.
- Build a chat-based UI interface in React that allows users to interact with the agent using natural language.
- Ensure that all file access is restricted to a specific directory, and that certain actions require user approval.

The project will be considered successful if the system can correctly locate files, read file contents, generate new files, and that the returned data is accurate.

Project Deliverables

- MCP Server
- Chat based UI
- AI agent that assists in file system interaction
- Documentation for use of the system

Measures / Evaluation

The success of this project will be measured by how well the system can safely and correctly work with local files, and how easy it is for users to interact with it.

- **Usability** will be evaluated by how easy the chat interface is to use. This includes how clearly the agent explains what it is doing and how simple it is for users to approve or deny file actions.

- **Performance** will be measured by how quickly and accurately the system can find files, read file contents, and save new files inside the allowed directory.
- **Verification and validation** will be done through testing of the MCP tools, both on their own and when used by the AI agent. Test cases will check that the correct files are found, the correct data is returned, and file operations work as expected.
- **Risk and security** will be evaluated by confirming that file access is limited to a specific folder, and that all write actions require user approval. Logs will be reviewed to make sure all actions are recorded.
- **CIAA (Confidentiality, Integrity, Availability, and Auditing):**
 - Confidentiality is maintained by restricting access to a single directory
 - Integrity is protected by requiring approval before files are changed
 - Availability is supported by keeping the system lightweight and able to run locally
 - Auditing is handled through logging of all tool calls and agent actions

These measures will be used to show that the system meets its goals and provides a safe and practical way for AI to work with the file system.

Transition / Adoption

This project will produce a working prototype that shows how MCP can be used to work with local files. The project can be used as a reference for future MCP-based work. The system may be shared as an open-source example so others can learn from it or build on it. The ideas could also be used in other projects that need controlled file access. Long-term use is outside the scope of this course, but the system is designed so it can be extended after the class.

Approach / Methodology

Our team will complete this project in phases so we can build the core features first and work on the additions after. This approach will help us stay organized and make sure each part is functional through the stages. Each Phase will be tested after it is built.

Phase 1: Build MCP Server with Fast MCP. The MCP server will be built first providing file system tools such as searching for files, reading file contents, and writing files.

Phase 2: Build AI Agent. We will implement an AI agent. The agent may be built from scratch or configure using a predefined LLM depending on the timeline. This agent will interact with users and take requests, select the MCP tools, and return results. Any changes made require user approval.

Phase 3: Running the System Locally. We will run the agent and MCP server as local services in Docker and connect them to the React chat interface. The UI will send user requests to the agent service, and the agent will call MCP tools to interact with the file system.

Resources

Resources we are planning to use are Fast MCP in Python to develop the MCP server; Docker will also be used to run the MCP server and other components in a local environment. Ollama will be utilized for AI to run an LLM, and a React based UI will be developed for the front end to allow the user to use a chat interface to interact with the server.

Risks and Opportunities

Risks

- If an agent is given complete access to the file system, then files not related to the task could be modified unintentionally.
Solution: File access will be limited to a single directory with the user's permission.
- If an agent returns incorrect information from files, then users can receive inaccurate results.
Solution: Users will review extracted information and give approval before any changes.
- If the chat interface does not clearly show what the system plans to do, then users may approve actions they did not fully understand.
Solution: The interface will display a clear summary of each proposed action before asking for approval.

Opportunities

If the system proves useful for file-based tasks, it may continue to be used after the course ends. To ensure this, the system will be designed around everyday use cases, and the server will be tested to ensure function.

Key Milestones

1. Developing the MCP server (2/20/26)
2. Connecting the server to an AI agent (3/20/26)
3. Developing React UI (4/19/26)
4. Test functionality (4/25/26)

Measures that will be taken are weekly meetings to make sure milestones are being accomplished and testing code functionality often.

Use of AI

AI will be used to help understand user requests and assist with file-related tasks using the Model Context Protocol. A local language model run with Ollama will support natural language interaction. The AI agent will not have direct access to the file system. All file actions will go through MCP tools that limit access to a specific folder and require user approval for file changes. This keeps the AI in an assisting role. To reduce risk, file access is restricted; write actions require approval. This ensures that the AI is used safely while users remain in control.

We will also be using AI as a peer programmer to help with code and test generation. We will be verifying the code created via code reviews and testing.

Project Team

This team will have shared responsibilities across system design, development, and integration. Joshua and Maurisa will focus on developing the MCP server. Their work will include defining MCP tools, implementing controlled file system access, and ensuring safety measures such as directory restrictions and approval requirements are enforced. Their academic focus and interest are in system design, backend development, and secure software integration.

Heng Yu will focus on exploring and implementing the AI-assisted component of the project. This includes evaluating approaches for using an AI agent with MCP tools, configuring the agent to interpret user requests, and supporting file-related tasks while maintaining safety and user control.

The team will collaboratively work on setting up the Docker environment to ensure the system runs consistently across machines. Development of the chat-based user interface using React will also be shared among team members to support integration and ensure the interface aligns with system functionality.

Maurisa Dacosta: is a graduate student in Computer Science with a background in scientific and technical problem solving and experience working with software systems and data-driven projects. Has a strong interest in data science and AI assisted systems.

Joshua Joubert: is a graduate student in Computer Science with a background in software engineering. Josh has been working as a professional software developer for about 20 years. He has a strong interest in AI and MCP.

Hengyu Liu: is a graduate student in Computer Science with a background in mechanical engineering. He has some interest in AI technology and likes to explore implementing some novel tech stacks into this system.

Future Vision

This project is a prototype that shows how MCP can connect AI-assisted tools with a local file system. Due to the timeline, the focus is on proof of concept rather than a full system. If time permits, there can be space to expand the system's functionality to AI-assisted process management. In the future, this approach could be expanded to support more file tasks or be applied to other systems that require controlled access.